LinkLab: A Data Link Layer Simulator with Error-Detection Benchmarking

Oct 26, 2026 · @Kaavin

Computer Networks individual project report · Kaavin Nitin Khatri · Computer Engineering · L.J. Institute of Engineering and Technology

Abstract

LinkLab is a browser-based simulator of the data link layer that joins framing, error detection and flow control into one working frame pipeline, and then measures how well each technique performs. A message is split into frames that carry an 8-bit sequence number and a CRC-8 checksum, are bit-stuffed inside HDLC flags, pass through a simulated noisy channel, and are checked and acknowledged by the receiver, which requests retransmission of any corrupted frame. The project implements bit and byte stuffing, parity (VRC), LRC, CRC and a 16-bit checksum, and the Stop-and-Wait, Go-Back-N and Selective Repeat protocols. Randomised experiments show that CRC-8 detected every burst error of up to 8 bits while VRC caught only 60.7% of them, and that at 50% loss Go-Back-N needed 6.93 transmissions per delivered frame against 4.02 for Selective Repeat. A companion Cisco Packet Tracer lab of two campuses joined by a serial WAN link shows the same ideas on real device models: Ethernet frames and their FCS, MAC addressing and learning, HDLC and PPP framing, and the TCP sliding window.

1. Introduction

1.1 Background

The data link layer moves frames reliably across a single physical link. It decides where each frame begins and ends (framing), notices when bits have been corrupted on the way (error detection), and controls how fast the sender transmits and what happens when a frame is lost (flow and error control). Every Ethernet, Wi-Fi and serial link in use today relies on these three functions working together on every frame.

1.2 Problem statement

In coursework these functions are usually studied as separate exercises: one program for bit stuffing, another for CRC, a third for Go-Back-N. Each program shows that an algorithm runs, but not how the pieces fit together on a real link, and not how well each technique actually performs against errors. Students also rarely see where these mechanisms appear inside real network traffic.

1.3 Objectives

1. Implement framing, error detection and flow control as a single frame pipeline that a user can run end to end.
2. Model a noisy channel that flips bits and loses frames and acknowledgements independently.
3. Measure the detection rate of each error-detection method, and the efficiency of each ARQ protocol, over thousands of randomised trials.
4. Demonstrate the same concepts on real Cisco device models in Packet Tracer.

1.4 Scope

The project covers the following items from the college list of project definitions: bit stuffing (#6), byte stuffing (#8), parity finding (#10, #13), VRC, LRC, CRC and checksum (#3, #4, #9, #14, #37), flow control (#7, #34), Go-Back-N and Selective Repeat (#15, #36), and framing in general (#35). Routing with OSPF is used in the Packet Tracer lab to join the two networks. Error correction (for example Hamming codes) and medium access control are outside the scope.

2. Theory

2.1 Framing

A receiver sees a continuous stream of bits and must find where each frame starts and ends. Bit-oriented protocols such as HDLC mark both ends with the flag 01111110. Because the flag is the only pattern with six consecutive 1s, the sender inserts a 0 after every five consecutive 1s in the data (bit stuffing), and the receiver removes the 0 that follows any five 1s. Six 1s inside a frame therefore always signal a framing error.

Byte-oriented protocols use a FLAG byte (here ~, 0x7E) and an ESC byte (}, 0x7D). Any FLAG or ESC byte inside the data is preceded by an ESC byte (byte stuffing), and the receiver drops each ESC and keeps the byte after it.

Technique	Data	Transmitted frame
Bit stuffing	11111111	01111110 111110111 01111110
Byte stuffing	A ~ B } C	~ A } ~ B } } C ~

2.2 Error detection

All four methods add redundant bits computed from the data; the receiver recomputes them and rejects the frame if they disagree.

- **Parity and VRC (vertical redundancy check).** One parity bit per character makes the number of 1s even (even parity). It detects any odd number of flipped bits in a character, but misses any even number.
- **LRC (longitudinal redundancy check).** A whole parity character is computed over each bit column of the block. It misses two flips in the same column of different characters.

- **CRC (cyclic redundancy check).** The data, followed by r zeros, is divided by a generator polynomial using modulo-2 (XOR) arithmetic, and the r-bit remainder is appended. The full codeword is then exactly divisible by the generator, so any non-zero remainder at the receiver means an error. A CRC with r check bits detects all single-bit errors, all burst errors of length r or less, and misses longer bursts with a probability of about 2^−r. This project uses CRC-8 with generator $x^8 + x^2 + x + 1$ (100000111).
- **Checksum.** The data is added as 16-bit words in ones' complement arithmetic (with end-around carry) and the complement of the sum is sent. The receiver adds every word, including the checksum, and accepts the frame only if the result is FFFF.

Worked CRC example for the message "NET": data 01001110 01000101 01010100, generator 100000111, remainder 00011011, so the codeword sent is 01001110 01000101 01010100 00011011.

2.3 Flow and error control (ARQ)

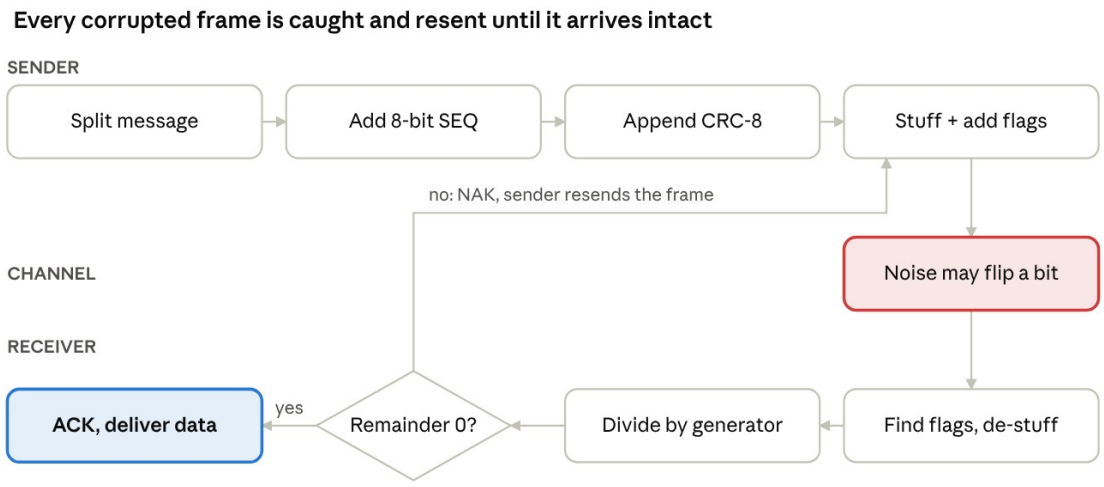
Automatic Repeat reQuest protocols retransmit frames that are lost or corrupted, using acknowledgements (ACKs) and timeouts.

- **Stop-and-Wait.** The sender transmits one frame and waits for its ACK before sending the next. It is simple but leaves the link idle while waiting.
- **Go-Back-N.** The sender may transmit up to N frames before needing an ACK. The receiver accepts frames only in order and sends cumulative ACKs. When a frame is lost, the sender times out and retransmits that frame and every frame after it. With m-bit sequence numbers the window is at most 2^m − 1.
- **Selective Repeat.** The receiver buffers frames that arrive out of order and acknowledges each one individually, so the sender retransmits only the frames that were actually lost. The window is at most 2^(m−1).

3. System design

3.1 Frame pipeline

LinkLab passes every frame through the same eight stages a real link uses, with a stop-and-wait retransmission loop around them.



LinkLab frame pipeline · 8 stages, 1 decision

A frame that fails the CRC check, or whose stuffing is broken by noise, is rejected and the sender retransmits it, so the received message always matches the original.

3.2 Frame format

Field	Size	Purpose
Flag	8 bits (01111110)	Marks the start of the frame
Sequence number	8 bits	Orders frames and matches ACKs
Payload	1 to 8 bytes (chosen by the user)	The message data
CRC	8 bits	Error detection, generator $x^8 + x^2 + x + 1$
Flag	8 bits (01111110)	Marks the end of the frame

Everything between the two flags is bit-stuffed before transmission.

3.3 Module structure

- **Framing module:** bit stuffing and de-stuffing with HDLC flags; byte stuffing with FLAG and ESC bytes.
- **Error-detection module:** encoding and checking for VRC, LRC, CRC and the 16-bit checksum, plus an error injector for single, double, triple and burst errors.
- **ARQ module:** event-by-event simulations of Stop-and-Wait, Go-Back-N and Selective Repeat with independent frame and ACK loss.
- **Experiment module:** randomised benchmarks that run the other modules thousands of times and summarise the results.

4. Implementation

4.1 Tools

Part	Technology	Purpose
Simulator	HTML, CSS and JavaScript in one file	Runs in any browser, online or offline
Network lab	Cisco Packet Tracer 9.0.1	Real device models: 1941 routers, 2960 switches
Reference version	C (gcc)	The same algorithms as a console program

4.2 Simulator modules

The simulator is organised into five tabs, each backed by the functions listed.

Tab	What the user does	Main functions
Pipeline	Types a message, sets bytes per frame and noise level, presses Transmit	bitStuff, bitDestuff, encode, check
Framing	Types text or a bit string and sees stuffing live	bitStuff, byteStuff, byteDestuff
Error detection	Picks a method and generator, clicks bits to flip them	encode, check, mod2div, csum16, inject
Sliding window	Picks a protocol, window, loss rate and seed, then plays the run	simGBN, simSR
Experiments	Runs the randomised benchmarks	encode, check, inject, simGBN, simSR

4.3 Key algorithms

- **Modulo-2 division.** For each position where the working bit is 1, the generator is XORed into the next g bits. The last g − 1 bits are the remainder. The same routine computes the CRC at the sender (on data plus zeros) and checks it at the receiver (on the full codeword).
- **Checksum.** Words are summed with sum = (sum & 0xFFFF) + (sum >> 16) after every addition, which implements end-around carry.
- **Go-Back-N.** Each round, the sender transmits every frame from the window base up to base + N. The receiver accepts only the expected frame and returns a cumulative ACK. At the end of a round the window slides to the highest ACK received, and any unacknowledged frames are resent from the base.
- **Selective Repeat.** Each frame has a state: unsent, sent, or acknowledged. The receiver buffers out-of-order frames and delivers a run of frames to the network layer as soon as the gap is filled. Only frames still unacknowledged at the end of a round are resent.

4.4 Noisy channel model

The channel uses a seeded pseudo-random generator, so a run can be repeated exactly by reusing its seed. In the Pipeline, each transmission is corrupted with the chosen probability by flipping one random bit between the flags. In the ARQ simulations, every frame and every ACK is lost independently with the chosen probability. The error-detection experiment injects single-bit, two-bit, three-bit and burst errors; a burst of length L always flips its first and last bits and flips each bit between them with probability one half.

5. Packet Tracer lab

The Packet Tracer file LinkLab.pkt builds two LANs, one per campus, joined by a serial WAN link. It shows the data link layer inside real traffic in Simulation mode.

5.1 Addressing

Device	Interface	IP address	Gateway
R1 (1941 + HWIC-2T)	Gi0/0 to SW1	192.168.10.1 /24	–
R1	Se0/1/0 to R2 (DCE, clock 64000)	10.0.0.1 /30	–
R2 (1941 + HWIC-2T)	Gi0/0 to SW2	192.168.20.1 /24	–
R2	Se0/1/0 to R1	10.0.0.2 /30	–
PC1, PC2, PC3	Fa0 to SW1	192.168.10.11 to .13 /24	192.168.10.1
PC4	Fa0 to SW2	192.168.20.11 /24	192.168.20.1
Server0 (web)	Fa0 to SW2	192.168.20.10 /24	192.168.20.1

5.2 Configuration

Both routers run OSPF process 1 in area 0, advertising their LAN and the 10.0.0.0/30 link. The serial link starts with Cisco’s default HDLC encapsulation. The switches are given hostnames and banners, and Server0 hosts a LinkLab web page over HTTP.

```
! R1
interface gigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
interface serial0/1/0
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000
 no shutdown
router ospf 1
 network 192.168.10.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
```

5.3 Demonstrations

Demo	What it shows	Syllabus link
Ethernet frame	Preamble, addresses, type and FCS fields of a frame	CRC
ARP and OSPF	Broadcast, unicast and multicast MAC addresses	MAC addressing
MAC table	A switch learning addresses and forwarding instead of flooding	Switching
HDLC vs PPP	The link fails when the two ends use different framing, then recovers	Framing
TCP	Sequence number, acknowledgement number and window fields	Sliding window

5.4 Physical view

In the Physical workspace, Home City contains two buildings. Campus A holds Wiring Closet A (R1 and SW1 in a rack) and PC1 to PC3. Campus B holds Wiring Closet B (R2, SW2 and Server0) and PC4. The serial WAN cable runs between the two buildings on the city map.

6. Results and analysis

6.1 Error-detection capability

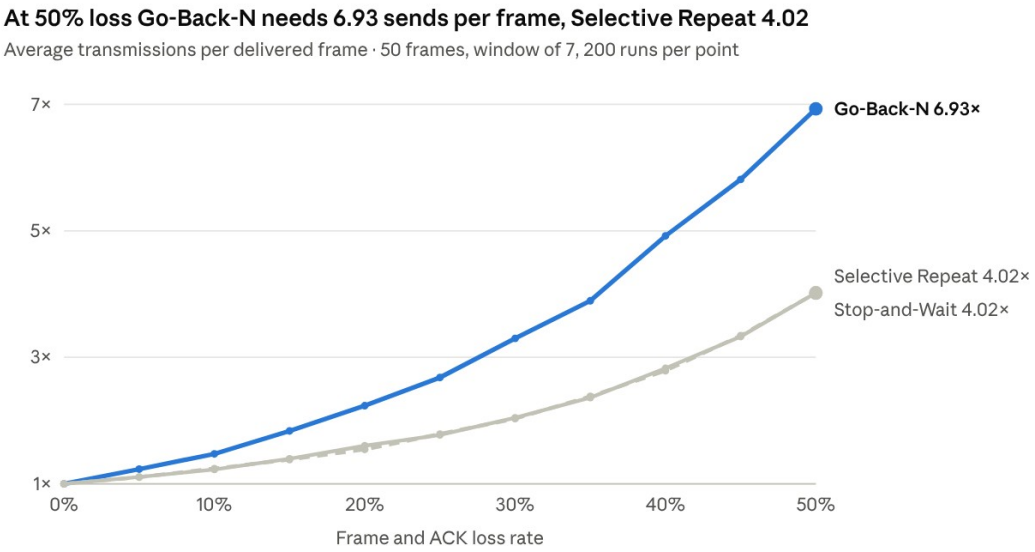
CRC-8 caught every single-bit, three-bit and short burst error; VRC caught only 60.7% of bursts up to 8 bits long. Each cell below is the share of corrupted frames that the receiver rejected, over 2,000 random 16-byte frames.

Error pattern	VRC	LRC	CRC-8	Checksum-16
Single-bit	100.00%	100.00%	100.00%	100.00%
Two-bit	94.80%	89.55%	99.80%	97.40%
Three-bit	100.00%	100.00%	100.00%	99.80%
Burst ≤ 8 bits	60.70%	100.00%	100.00%	100.00%
Burst of 12	81.35%	99.70%	99.60%	100.00%
Burst of 20	88.60%	99.80%	99.80%	100.00%

- **VRC** misses any even number of flips inside one character. It scores 94.8% on two-bit errors only because two random flips usually land in different characters.
- **LRC** misses two flips in the same bit column of different characters, which is why it is the weakest method on two-bit errors.

- **CRC-8** detects every burst of 8 bits or fewer, as theory predicts for 8 check bits, and misses roughly 1 longer burst in 256 (99.6%).

6.2 Protocol efficiency



LinkLab protocol experiment · loss 0% to 50% in 5% steps

At low loss the three protocols are close, but Go-Back-N degrades fastest because every lost frame or ACK costs a whole window of retransmissions. Selective Repeat uses about as few transmissions as Stop-and-Wait, whose curve it overlaps, while still keeping up to seven frames in flight, so it is the best choice on a lossy link.

7. Testing

Every check passed. The simulator's algorithms were checked against hand calculation and an independent implementation, and the Packet Tracer lab was tested end to end.

Test	Expected	Observed	Result
CRC-8 of "NET"	Remainder 00011011	00011011 in both the simulator and an independent check	Pass
Bit stuffing round trip	De-stuffed data equals the original	Recovered exactly for text and bit-string inputs	Pass
Byte stuffing round trip	De-stuffed data equals the original	"A~B}C" recovered exactly, 2 ESC bytes inserted	Pass
Pipeline with 60% noise	Every corrupted frame caught and resent	Message delivered intact after retransmissions	Pass
Ping PC1 to	Replies across the WAN	4 of 4 replies, 0% loss, about 13 ms	Pass

Test	Expected	Observed	Result
Server0			
Web page over HTTP	LinkLab page loads on PC1	Page loaded from 192.168.20.10	Pass
OSPF on R1	Route to 192.168.20.0/24	O 192.168.20.0/24 via 10.0.0.2, neighbour state FULL	Pass
Simulation mode	Packet crosses both LANs and the WAN	PC1 → SW1 → R1 → R2 → SW2 → Server0 and back, status Successful	Pass

The first ping of a session loses one reply while the PC resolves its gateway's MAC address with ARP, which is expected. Packet Tracer shows the Ethernet FCS field as 0x00000000 because it does not compute a real CRC; the simulator is where the CRC calculation itself is demonstrated.

8. Conclusion and future scope

8.1 Conclusion

LinkLab shows the data link layer as one connected system rather than a set of separate exercises. Running framing, error detection and ARQ together on the same frames makes it clear why each is needed. The experiments turn the theory into numbers: CRC-8 detected every burst of up to 8 bits, exactly as its 8 check bits predict, while parity missed 39% of them because it cannot see an even number of flips in a character; and on a link losing half its frames, Selective Repeat needed 42% fewer transmissions than Go-Back-N. The Packet Tracer lab confirms that the same mechanisms appear in real Ethernet, HDLC, PPP and TCP traffic.

8.2 Future scope

- Add Hamming code to move from detecting single-bit errors to correcting them.
- Model propagation delay and bandwidth so that throughput and link utilisation can be plotted, not only transmission counts.
- Show sequence-number wrap-around and the window-size limits 2^m − 1 and 2^(m−1) in action.
- Add medium access control protocols such as CSMA/CD and slotted ALOHA.

References

1. B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013. Chapters on data link control and error detection.
2. A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011. Chapter 3, The Data Link Layer.
3. W. Stallings, *Data and Computer Communications*, 10th ed., Pearson, 2013.
4. ISO/IEC 13239, *High-level data link control (HDLC) procedures*.

5. W. Simpson, RFC 1661, *The Point-to-Point Protocol (PPP)*, IETF, 1994.
6. Cisco Networking Academy, *Cisco Packet Tracer* documentation.